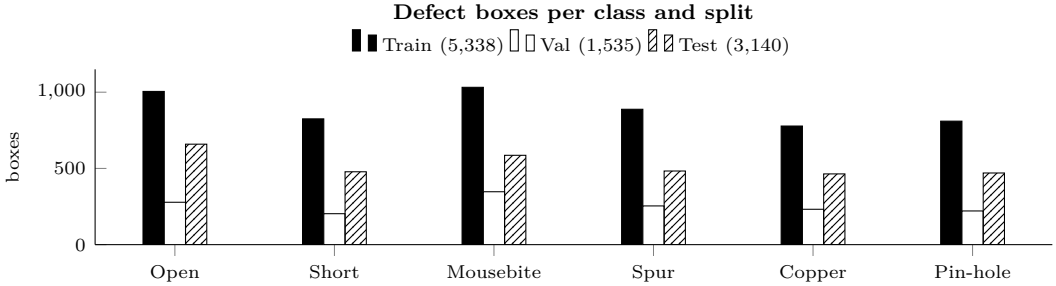
PCB Defect Detection Under Image Misalignment

Data Report

Basma Kurany (40374114) · COEN 6314, Concordia University · November 2026

Summary. DeepPCB is a supervised *object-detection* dataset: each defect must be located (bounding-box *regression*) and labelled with one of six classes (*classification*). We added **controlled misalignment**: every validation and test pair is rendered under 10 seeded conditions (as supplied; 1/2/3/5 px shifts in x or y; a sub-pixel shift with $\leq 0.5°$ rotation), boxes moving with the image, to compare YOLOv10-Nano (trained with and without shift augmentation) against a registration-and-differencing baseline. **Statistics** (dashboard): **1,500** pairs, **800/200/500** split, **0–5 px** shifts plus a $\leq 0.5°$ residual, **9/9** checks passed. **Precautions:** splits are made by board group before augmentation (no board shared by train and val); both methods read identical shifted files; an 8 px border is excluded and 258 edge boxes are ignored in every condition. **Considerations:** 123 pairs arrive misaligned, 168 test pairs share boards with training and 3 copy train pairs, so clean and unseen-board subsets are also reported; most defects are synthetic.

1,500	**800/200/500**	**0–5 px**	**9/9**
image pairs	train / val / test	misalignment range	checks passed

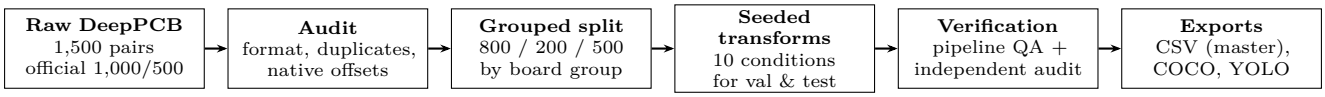


1 Composition: what the data contains

DeepPCB [1] provides 1,500 pairs of 640×640 near-binary images, each a defect-free *template* (reference) and an *inspected* image of the same PCB region, pre-aligned by the authors. Each pair carries 1–15 boxes (mean 6.7) from six near-balanced classes (largest/smallest ratio 1.33). Defects are small (median 35×33 px, 0.3 % of the image) among many long track edges—the edges misalignment turns into false differences. The pairs come from 11 board *groups*, the only board identifier.

2 Preparation pipeline

All outputs are rebuilt from the untouched download by `scripts/run_all.sh`; seed (6314, pair ID) makes them reproducible.



3 Splits and board leakage

Crops of the same board look alike, so splitting them across partitions would leak information. DeepPCB has no board ID; its 11 *group* folders are used as the board unit, and all images of a group are kept together.

- **Train vs. validation:** about 20 % of trainval is held out *before* any augmentation by assigning whole groups. Among all group subsets holding 17–23 % of trainval, group `00041` (200 pairs) has the class mix closest to trainval ($L_1 = 0.059$). Train holds the other six groups; the independent audit confirms no group appears in both. All ten shifted copies of a pair inherit its split.
- **Is a group really a board?** Yes: all 40 confirmed near-duplicates lie inside a single group.
- **Test:** the official list is kept, but it mixes boards: 168 test pairs come from 7 groups also in trainval (21 from the validation group) and 3 are near-copies of train pairs. The other 332 come from four groups never seen in training, so test results are reported on all 500 pairs, on the 332 unseen-board pairs, and without the 3 duplicates; similar numbers show leakage is not inflating them.

4 Controlled misalignment

The reference is never modified; the inspected image and its boxes always move together. Each validation and test pair is given ten conditions, and the baseline and detector read the same files:

- `s0`: the pair as supplied;
- `s1x`…`s5y`: integer shifts of 1, 2, 3 and 5 px along x or y (random sign), copied pixel-exactly with no interpolation;
- r (residual): a 0.25–1.5 px sub-pixel translation plus a rotation within $\pm 0.5°$ (bilinear), which whole-pixel registration cannot undo; rotated boxes become the axis-aligned envelope of their corners.

This gives $200 \times 10 = 2{,}000$ validation and $500 \times 10 = 5{,}000$ test images (46,750 boxes). Outside the labelled boxes, pixels differing from the template rise from 1.8 % of the image as supplied to 6.7 % at 5 px, i.e. from 6 to 16 times the defect pixels. **Training does see misalignment:** the non-augmented run uses the 800 pairs as supplied (75

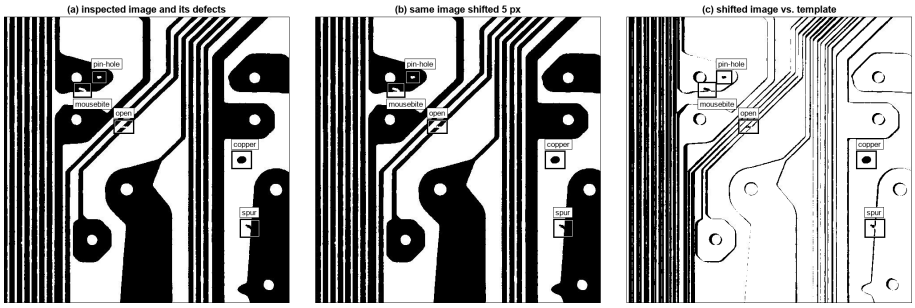


Figure 1: Test pair `12300244`: (a) inspected image and its labelled defects; (b) shifted 5 px with its boxes; (c) the shifted image vs. the unchanged template, black = differing pixels. The shift turns every track edge into a false difference among the few real defects—what the detector must learn to ignore.

already natively offset), and the augmented run adds one random 1–5 px shift and one residual copy of every pair (2,400 images, separate seed stream; Section 7). **Edges:** uncovered pixels lie in an 8 px border that evaluation excludes; boxes originally within 14 px of an edge (258, 2.6 %) are *ignored* in every condition: neither hit, miss nor false positive.

5 Known issues and how they are handled

Issue	Evidence	Handling
Native misalignment	123 "aligned" pairs (75/8/40 train/val/test) offset 1–8 px, mostly 2–3 px (XOR area cut $\geq 1.5\times$).	Offset stored; also report the 1,377 clean pairs (460 test).
Near-duplicates	dHash + pixel check: 40 confirmed, all within one group, none across train/val; 3 test pairs copy a train pair.	Flagged per pair; report test with and without them.
Board overlap	7 of 11 test groups also occur in trainval (168 pairs).	Unavoidable with the official split; also report the 332 unseen-board test pairs.

6 Verification

Two independent sets of checks were run: the pipeline's QA scripts, and a separate audit that re-decodes the raw files with a different library (Pillow/SciPy, not OpenCV) and shares no pipeline code. The only difference is one colour JPEG (`44000020`) whose grey conversion differs between decoders; its shifted copies still match its own `s0` exactly. Visual sheets like Fig. 1 were checked for every condition.

Check	Result
Pair lists, splits, `boxes.csv` match raw	1500/1500
References identical to raw templates	1500/1500
`s0` identical to raw (independent decoder)	1499/1500
Integer shift = `s0` moved by (dx, dy)	5600/5600
Box count, class, offset, ignore flag	52088/52088
Non-ignored boxes inside valid region	41220/41220
Residual images reproduced from parameters	700/700
COCO files consistent with CSV	21/21
YOLO images with labels, zip CRC intact	10200/10200

7 Intended use and formats

The master record is a set of CSV files in `index/` (below). They hold annotations and metadata for splitting, evaluation and error analysis; the detector's input features are the image pixels themselves.

CSV file	One row per	Fields recorded
`boxes.csv`	original box (10,013)	pair, group, official split, raw corners x_1, y_1, x_2, y_2, class, width, height
`splits.csv`	source pair (1,500)	pair, group, split, raw paths, box count, native offset + flag, duplicate flag
`images.csv`	inspected image (7,800)	image, pair, split, condition, paths, applied dx, dy, rotation, magnitude, box/ignore counts
`labels.csv`	box per image (52,088)	image, pair, split, condition, transformed x_1, y_1, x_2, y_2, class (1–6), ignore flag

Why COCO: 21 COCO files (one per split and condition) mirror the CSVs because COCO is the standard detection format, read by torchvision, Detectron2 and MMDetection (so Fast R-CNN or SSD can be swapped in), and its `iscrowd` flag makes `pycocotools` treat ignore boxes as don't-care. **YOLO:** each pair is one 3-channel image (reference, inspected, |difference|); the augmented run trains for a third of the epochs (equal steps), colour augmentation off. Thresholds are set on validation only; test is evaluated once.

8 Proposal V4 requirements

Requirement (Section 2 of the proposal)	Implementation	
Official 1,000/500 split; ∼20 % val before augmentation	800/200/500; val chosen before any transform	✓
Same board and all shifted variants in one partition	Whole groups for train/val; shifted copies inherit split	✓
Check cross-partition near-duplicates	40 confirmed, none train/val; 3 test pairs flagged	✓
Reference fixed; image and boxes shifted 0/1/2/3/5 px in x or y	Conditions `s0`–`s5y`, pixel-exact	✓
Residual set: 0.25–1.5 px and rotation up to 0.5°	Condition r, one per val/test pair	✓
Identical seeded shifts for both methods	Shared files, seed (6314, pair ID)	✓
Exclude padded borders; ignore truncated defects consistently	8 px border; 258 boxes ignored in every condition	✓
Visually check transformed labels	QA sheets for every condition (Fig. 1)	✓

Reference. [1] S. Tang *et al.*, "Online PCB defect detector on a new PCB defect dataset," *arXiv:1902.06197*, 2019.